Лабораторная работа: векторный вход и бинарная классификация

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Содержание

Как пользоваться этой работой **1**
Маршрут чтения и выполнения . . . 1
Индексы в формулах и имена Python . . . 1

Учебный контракт **2**
Зачем OR, AND и XOR в этой лабораторной . . . 2

Подготовка: скалярный граф и вектор признаков **2**
Импорт и сравнение чисел . . . 2
Полный класс Value . . . 3
Короткая проверка прежнего механизма . . . 5
Почему вектор не требует нового класса Value . . . 5
Общие параметры всех объектов датасета . . . 6
Почему sigmoid и binary cross-entropy образуют пару . . . 8

Этап 1. Один sigmoid-нейрон с векторным входом **8**
Прямой проход на OR . . . 9
Градиенты одного нейрона . . . 9
Обучение OR . . . 10

Этап 2. Один скрытый нейрон и один выходной нейрон **10**
Градиенты через скрытый нейрон . . . 11
Обучение AND . . . 12
Зачем после AND нужен XOR . . . 13

Этап 3. Скрытый слой из двух нейронов и XOR **13**
Градиенты XOR до обучения . . . 14
Обучение XOR . . . 15

Диагностика **15**
Что намеренно не реализуется . . . 16

Чек-лист перед сдачей **16**

Вопросы для самопроверки **16**

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Как пользоваться этой работой

Эта лабораторная продолжает работу «Скалярное автоматическое дифференцирование и обучение одного нейрона». Здесь механизм `Value` по-прежнему работает только со скалярами. Новым является не тип вершины графа, а модель: один объект датасета имеет вектор признаков `features = [x_1, x_2]`, а нейрон вычисляет сумму нескольких скалярных произведений.

Откройте `student_vector_binary_classification_lab.ipynb`. В каждой пустой code-ячейке есть идентификатор. Копируйте в неё только полный блок с тем же идентификатором из этого guide. Не меняйте порядок запуска ячеек. Образец `teacher_vector_binary_classification_lab` содержит те же ячейки и предназначен для проверки после самостоятельной работы.

Все вычисления используют только стандартную библиотеку Python.

Маршрут чтения и выполнения

Не нужно сначала запоминать всю теорию целиком. Читайте выданный теоретический материал и выполняйте notebook в следующем порядке.

1. Перед подготовкой и первым этапом OR прочитайте «Минимальное повторение перед началом» и разделы 1–3: они восстанавливают скалярный `Value`, вводят датасет, расширенный вход, логит, sigmoid и BCE.
2. Перед ячейкой `or_gradients` прочитайте разделы 4–5: они выводят градиент логита, градиенты весов и показывают тот же результат через reverse-mode.
3. Перед этапом AND прочитайте раздел 6: он вводит композицию скрытого tanh-нейрона и sigmoid-выхода.
4. Перед этапом XOR прочитайте разделы 7–8: они объясняют ограничение одного скрытого нейрона и роль двух скрытых характеристик.
5. После успешного выполнения всех трёх этапов прочитайте раздел 9: это математическое обобщение уже проверенных скалярных вычислений до векторов, матриц и MLP.

Выполняйте code-ячейки guide сверху вниз и не меняйте их порядок.

Индексы в формулах и имена Python

В общих формулах i, j, h и k — свободные индексы: они фиксируют соответственно объект, входную координату или вес, скрытый нейрон и выходную либо скрытую координату. Греческие α, β и μ — немые индексы явных свёрток: по ним стоят знаки $\sum$ или $\prod$. Немой индекс можно согласованно переименовать; свободный индекс определяет компоненту результата и должен сохраняться в равенстве.

Математический индекс	Роль	Представление в Python
α	немая сумма по расширенным входным координатам	пара `weight, feature` в `affine`; `zip` выдаёт очередную пару без переменной `alpha`
β	немая сумма по расширенным скрытым состояниям на выходе	список `hidden` передаётся как `features` во второй вызов `affine`
μ	немая агрегация по объектам датасета	`for features, target in zip(inputs, targets)` в средней BCE
h	один фиксированный скрытый нейрон	одна строка `hidden_layer_weights` и соответствующий элемент списка `hidden`

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Математический индекс	Роль	Представление в Python
i, j, k	фиксированный объект или компонент результата формулы	смысл задаётся текущей парой данных или позицией списка; отдельные индексные переменные не нужны

Имена Python намеренно остаются английскими и семантическими. Они не копируют греческие буквы: `zip` и списки передают очередные значения, а математическая свёртка остаётся явной в формуле.

Учебный контракт

После работы вы должны уметь:

- развернуть $\sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha}$ в цикл скалярных операций `Value`, где w_0 — вес постоянной координаты 1;
- объяснить, почему один и тот же объект веса получает вклады градиента от всех объектов датасета;
- применить sigmoid и binary cross-entropy для бинарной классификации;
- обучить один sigmoid-нейрон на OR;
- проследить градиент через один скрытый нейрон с активацией tanh и sigmoid-выход;
- объяснить, почему один скрытый нейрон не решает XOR;
- обучить слой из двух скрытых нейронов для XOR.

Во всех трёх частях метка класса равна 0 или 1, а результат модели p интерпретируется как вероятность класса 1. Метка предсказания равна 1, если $p \geq 0.5$, и 0 иначе.

Зачем OR, AND и XOR в этой лабораторной

OR, AND и XOR здесь не прикладные целевые функции, а маленькие контролируемые датасеты. OR проверяет, что один sigmoid-нейрон обучает линейно разделимую зависимость. AND намеренно используется со скрытым нейроном, чтобы сделать наблюдаемым градиент через композицию двух аффинных операций и tanh, хотя один sigmoid-нейрон способен решить AND и без скрытого слоя. XOR служит тестом выразительности: его нельзя классифицировать одной линейной границей, зато два разных скрытых признака позволяют это сделать.

В реальной задаче вместо этих четырёх бинарных объектов будут вещественные векторы признаков и большое число меток. Но цикл `признаки → вероятность → BCE → backward() → обновление` и роль общих параметров остаются теми же. Полное математическое обоснование и связь с MLP даны в выданном теоретическом материале.

Подготовка: скалярный граф и вектор признаков

Импорт и сравнение чисел

В ячейку `imports_and_helper` вставьте:

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: imports_and_helper
import math


def assert_close(actual, expected, tolerance=1e-9):
    difference = abs(actual - expected)
    assert difference <= tolerance, (
        f"Ожидалось {expected}, получено {actual}, "
```


© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

```
        f"абсолютная ошибка {difference}"
    )
```

Полный класс Value

В этой работе вы не реализуете autograd заново. Перенесите в ячейку `value` полный эталонный класс из предшествующей лабораторной: он намеренно приведён целиком и не импортируется из соседней директории, поэтому notebook остаётся самостоятельным. Сначала ячейка `scalar_graph_refresh` ниже подтвердит на маленьком графе, что этот механизм уже работает; затем те же объекты `Value` будут применены к векторному входу. Не сокращайте класс и не меняйте его методы.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: value
class Value:
    def __init__(self, data, parents=(), op=""):
        self.data = float(data)
        self.grad = 0.0
        self.parents = parents
        self.op = op
        self._backward = lambda: None

    def __repr__(self):
        return (
            f"Value(data={self.data:.6g}, grad={self.grad:.6g}, "
            f"op={self.op!r})"
        )

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        result = Value(
            self.data + other.data,
            parents=(self, other),
            op="+",
        )

        def backward():
            self.grad += result.grad
            other.grad += result.grad

        result._backward = backward
        return result

    __radd__ = __add__

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        result = Value(
            self.data * other.data,
            parents=(self, other),
            op="*",
        )

        def backward():
            self.grad += result.grad * other.data
            other.grad += result.grad * self.data

        result._backward = backward
        return result

    __rmul__ = __mul__
```

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

```
    def __neg__(self):
        result = Value(-self.data, parents=(self,), op="neg")

        def backward():
            self.grad += result.grad * (-1)

        result._backward = backward
        return result

    def __sub__(self, other):
        return self + (-other)

    def __rsub__(self, other):
        return other + (-self)

    def __pow__(self, power):
        result = Value(
            self.data ** power,
            parents=(self,),
            op=f"pow({power})",
        )

        def backward():
            local_derivative = power * self.data ** (power - 1)
            self.grad += result.grad * local_derivative

        result._backward = backward
        return result

    def sin(self):
        result = Value(math.sin(self.data), parents=(self,), op="sin")

        def backward():
            self.grad += result.grad * math.cos(self.data)

        result._backward = backward
        return result

    def exp(self):
        result = Value(math.exp(self.data), parents=(self,), op="exp")

        def backward():
            self.grad += result.grad * result.data

        result._backward = backward
        return result

    def log(self):
        result = Value(math.log(self.data), parents=(self,), op="log")

        def backward():
            self.grad += result.grad * (1 / self.data)

        result._backward = backward
        return result

    def tanh(self):
        result = Value(math.tanh(self.data), parents=(self,), op="tanh")

        def backward():
            self.grad += result.grad * (1 - result.data ** 2)
```


© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

```
        result._backward = backward
        return result

    def backward(self):
        topo = []
        visited = set()

        def build(node):
            if node not in visited:
                visited.add(node)
                for parent in node.parents:
                    build(parent)
                topo.append(node)

        build(self)

        for node in topo:
            node.grad = 0.0

        self.grad = 1.0

        for node in reversed(topo):
            node._backward()
```

Короткая проверка прежнего механизма

До векторного входа зафиксируем исходную точку. Уже известно: `Value` строит скалярный DAG и складывает вклады в `grad`. Теперь проверяем: повторное использование одного листа a в $J = a^2 + a$ должно дать две части от a^2 и одну часть от слагаемого a. Наблюдаемый результат: при $a = 2$ ячейка печатает $J = 6$ и $\frac{\partial J}{\partial a} = 5$. Это тот же пример, который разобран в минимальном повторении теории.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: scalar_graph_refresh
a = Value(2.0)
objective = a * a + a
objective.backward()

assert_close(objective.data, 6.0)
assert_close(a.grad, 5.0)
print("Скалярный Value готов: J =", objective.data, ", dJ/da =", a.grad)
```

Почему вектор не требует нового класса Value

Пусть один объект имеет два обычных числовых признака `features = [x_1, x_2]`. Сырые данные не содержат единицу. Функция `affine` сама создаёт внутренний расширенный список `[1.0, *features]`, то есть $(1, x_1, x_2)$. Параметры `weights = [w_0, w_1, w_2]` являются долгоживущими объектами `Value`; w_0 — смещение, представленное как вес постоянной координаты. Функция `affine` строит выражение

$$z = w_0 \cdot 1 + w_1 x_1 + w_2 x_2.$$

Цикл Python выполняет три скалярные операции умножения и две операции сложения. Поэтому получается обычный скалярный DAG, в том числе с настоящим узлом $w_0 \cdot 1$. Число-признак и постоянная единица временно оборачиваются методом `Value.__mul__` в константные листья; они не входят в `parameters()` и не обновляются градиентным спуском.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Общие параметры всех объектов датасета

Один и тот же объект, например `weights[0]`, используется при построении графа каждого объекта датасета. После сложения потерь обратный проход обходит все эти графы и выполняет `+=` в одном поле `weights[0].grad`. Так возникает сумма вкладов по выборке.

Вставьте вспомогательные функции в `vector_helpers`.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: vector_helpers
def affine(weights, features):
    # Для фиксированного объекта i: o_i = sum_{alpha=0}^d w_alpha x_tilde[i, alpha].
    # zip передаёт очередную пару без Python-переменной alpha; сумма остаётся в
     ↪  формуле.
    augmented_features = [1.0, *features]
    assert len(weights) == len(augmented_features), (
        "Число весов должно включать вес постоянной координаты"
    )
    total = weights[0] * augmented_features[0]

    for weight, feature in zip(weights[1:], augmented_features[1:]):
        total = total + weight * feature

    return total


def sigmoid(logit):
    return (1.0 + (-logit).exp()) ** -1


def binary_cross_entropy(probability, target):
    target = float(target)
    assert target in (0.0, 1.0), "Метка бинарной классификации должна быть 0 или 1"
    return -(
        target * probability.log()
        + (1.0 - target) * (1.0 - probability).log()
    )


def mean_binary_cross_entropy(model, inputs, targets):
    assert len(inputs) == len(targets), "Число объектов и меток должно совпадать"
    assert len(inputs) > 0, "Нужен хотя бы один объект"
    total = Value(0.0)

    # Немая сумма по объектам mu: L = (1 / n) sum_mu ell_mu.
    for features, target in zip(inputs, targets):
        probability = model(features)
        total = total + binary_cross_entropy(probability, target)

    return total * (1.0 / len(inputs))


def predict_label(probability):
    return 1 if probability.data >= 0.5 else 0


def classification_accuracy(model, inputs, targets):
    predictions = [predict_label(model(features)) for features in inputs]
    correct = sum(
        prediction == target
        for prediction, target in zip(predictions, targets)
    )
    return correct / len(targets)
```


© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

```
def numerical_gradient(model, parameter, inputs, targets, step=1e-6):
    original_data = parameter.data
    parameter.data = original_data + step
    loss_plus = mean_binary_cross_entropy(model, inputs, targets).data
    parameter.data = original_data - step
    loss_minus = mean_binary_cross_entropy(model, inputs, targets).data
    parameter.data = original_data
    return (loss_plus - loss_minus) / (2.0 * step)
```

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Почему sigmoid и binary cross-entropy образуют пару

Здесь sigmoid строится только из скалярных операций существующего `Value`:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

Бинарная кросс-энтропия (англ. binary cross-entropy, BCE) — функция потерь для двух классов. Если модель дала вероятность p класса 1, то вероятность наблюдаемой правильной метки $y \in \{0, 1\}$ равна:

$$P(y \mid p) = p^y (1 - p)^{1-y}.$$

BCE — отрицательный логарифм этой вероятности:
Для одной вероятности p и метки y функция потерь равна

$$\ell = -y \log p - (1 - y) \log(1 - p).$$

Если $y = 1$, остаётся $-\log p$ и модель штрафуется за малую вероятность верного класса. Если $y = 0$, остаётся $-\log(1 - p)$ и штрафуется большая вероятность класса 1. BCE даёт большой штраф за уверенный неверный прогноз и гладкий градиент для `backward()`. Accuracy с порогом 0.5 для обучения не подходит: она не меняется при малых изменениях вероятности и не сообщает, насколько неверен прогноз. Полное вероятностное обоснование и вывод связи BCE с правдоподобием приведены в теоретическом материале.

Функция `numerical_gradient` меняет параметр только между двумя новыми прямыми проходами и обязательно восстанавливает исходное число. Она никогда не изменяет параметр между созданием конкретного `loss` и его `loss.backward()`.

Этап 1. Один sigmoid-нейрон с векторным входом

Уже известно: один скалярный `Value` умеет хранить число, родителей и накопленный градиент. Теперь проверяем: один и тот же механизм строит аффинную сумму для двух признаков и обучает три общих веса, включая w_0. Наблюдаемый результат этапа: loss OR станет меньше 0.1, а все четыре объекта будут классифицированы верно.

Модель первого этапа — логистический нейрон:

$$o_i = \sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha}, \qquad p_i = \sigma(o_i).$$

Для набора из n объектов optimizируется средняя binary cross-entropy:

$$L = \frac{1}{n} \sum_{\mu=1}^{n} \ell_\mu.$$

Создайте модель в ячейке `vector_neuron`.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: vector_neuron
class VectorBinaryNeuron:
    def __init__(self, weights):
        assert len(weights) > 1, "Нужны вес смещения и хотя бы один входной вес"
        self.weights = [Value(weight) for weight in weights]

    def __call__(self, features):
        logit = affine(self.weights, features)
        return sigmoid(logit)

    def parameters(self):
        return self.weights
```


© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

`parameters()` возвращает ссылки на объекты `Value`, а не их числа. Поэтому строка обновления `parameter.data -= learning_rate * parameter.grad` меняет именно веса, которые следующий прямой проход снова использует.

Прямой проход на OR

В ячейке `or_forward` проверьте форму данных и начальный прогноз. Начальные параметры специально не обучены: совпадение всех меток на этом шаге не ожидается.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: or_forward
OR_INPUTS = [
    [0.0, 0.0],
    [0.0, 1.0],
    [1.0, 0.0],
    [1.0, 1.0],
]
OR_TARGETS = [0, 1, 1, 1]

or_neuron = VectorBinaryNeuron(weights=[0.0, 0.1, -0.2])
or_probabilities = [or_neuron(features).data for features in OR_INPUTS]

assert_close(or_probabilities[0], 0.5)
assert_close(or_probabilities[1], 1.0 / (1.0 + math.exp(0.2)))
assert len(or_neuron.parameters()) == 3
assert all(isinstance(parameter, Value) for parameter in or_neuron.parameters())
print("Начальные вероятности OR:", or_probabilities)
```

Градиенты одного нейрона

Для одного объекта правило цепочки даёт простое выражение

$$\frac{\partial \ell_i}{\partial o_i} = p_i - y_i.$$

Следовательно, для координаты веса

$$\frac{\partial L}{\partial w_j} = \frac{1}{n}\sum_{\mu=1}^{n}(p_\mu - y_\mu)\tilde{x}_{\mu j}, \qquad j = 0, \dots, d.$$

Автоматическое дифференцирование не подставляет эту итоговую формулу вручную. Оно строит граф для каждого i, а затем точно складывает те же вклады в общих `Value`-параметрах. Проверьте это центральной разностью.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: or_gradients
gradient_or = VectorBinaryNeuron(weights=[0.0, 0.1, -0.2])
or_loss = mean_binary_cross_entropy(gradient_or, OR_INPUTS, OR_TARGETS)
or_loss.backward()

for parameter in gradient_or.parameters():
    expected_gradient = numerical_gradient(
        gradient_or,
        parameter,
        OR_INPUTS,
        OR_TARGETS,
    )
    assert_close(parameter.grad, expected_gradient, tolerance=1e-5)

assert any(abs(parameter.grad) > 1e-8 for parameter in gradient_or.parameters())
print("BCE для OR до обучения:", or_loss.data)
print("Градиенты OR:", [parameter.grad for parameter in gradient_or.parameters()])
```

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Обучение OR

Одна эпоха строит свежий граф всей средней потери, запускает `backward()`, затем обновляет долгоживущие параметры. Нельзя переносить обновление до `loss.backward()`: замыкания операций используют значения прямого прохода.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: train_or
learning_rate = 0.5
epochs = 300
trained_or = VectorBinaryNeuron(weights=[0.0, 0.1, -0.2])
or_losses = []

for epoch in range(epochs):
    loss = mean_binary_cross_entropy(trained_or, OR_INPUTS, OR_TARGETS)
    loss.backward()
    or_losses.append(loss.data)

    for parameter in trained_or.parameters():
        parameter.data -= learning_rate * parameter.grad

or_predictions = [trained_or(features).data for features in OR_INPUTS]
or_accuracy = classification_accuracy(trained_or, OR_INPUTS, OR_TARGETS)

assert or_losses[-1] < 0.1, or_losses[-1]
assert or_losses[-1] < or_losses[0]
assert or_accuracy == 1.0, or_accuracy
assert all(abs(parameter.data) > 0.1 for parameter in trained_or.parameters())
print("OR: loss в первой эпохе:", or_losses[0])
print("OR: loss в последней эпохе:", or_losses[-1])
print("OR: вероятности:", or_predictions)
print("OR: точность:", or_accuracy)
```

Для $j = 0$ в эту формулу подставляется $\tilde{x}_{i0} = 1$: нулевой вес является смещением без специального правила. Результат этапа: один нейрон провёл одну линейную границу и классифицировал OR. Его весовой список содержит постоянную координату и два реальных входа, но каждая базовая операция autograd остаётся скалярной.

Этап 2. Один скрытый нейрон и один выходной нейрон

Уже известно: один sigmoid-нейрон получает градиенты своих весов непосредственно от логита. Теперь проверяем: градиент проходит через выходной sigmoid, выходную аффинную сумму, скрытый tanh и скрытую аффинную сумму. Наблюдаемый результат этапа: центральная разность подтверждает градиенты пяти параметров, затем loss AND становится меньше 0.05 и точность равна 1.0.

AND выбран не потому, что ему архитектурно необходим скрытый нейрон. Его цель — проверить прохождение градиента через композицию скрытой и выходной частей модели.

Теперь вектор сначала поступает в скрытый нейрон:

$$h_i = \tanh\left(\sum_{\alpha=0}^{2} u_\alpha \tilde{x}_{i\alpha}\right),$$

а затем в выходной sigmoid-нейрон:

$$o_i = \sum_{\beta=0}^{1} v_\beta \tilde{h}_{i\beta} = v_0 + v_1 h_i, \qquad p_i = \sigma(o_i).$$

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Скрытый tanh выбран потому, что это гладкая нелинейная функция с известной локальной производной $1 - h_i^2$. Sigmoid выбран только для выхода, где число должно быть вероятностью класса 1.

Создайте модель в one_hidden_model.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: one_hidden_model
class OneHiddenBinaryClassifier:
    def __init__(
        self,
        hidden_weights,
        output_weights,
    ):
        assert len(hidden_weights) > 1, "Нужны вес смещения и хотя бы один входной вес"
        assert len(output_weights) == 2, "Выходу нужны вес смещения и вес скрытого
         ↪  состояния"
        self.hidden_weights = [Value(weight) for weight in hidden_weights]
        self.output_weights = [Value(weight) for weight in output_weights]

    def hidden_state(self, features):
        # Один фиксированный скрытый нейрон; affine сворачивает входные координаты
         ↪  alpha.
        hidden_logit = affine(self.hidden_weights, features)
        return hidden_logit.tanh()

    def __call__(self, features):
        hidden = self.hidden_state(features)
        output_logit = affine(self.output_weights, [hidden])
        return sigmoid(output_logit)

    def parameters(self):
        return self.hidden_weights + self.output_weights
```

Градиенты через скрытый нейрон

Для потери одного объекта обозначим локальную производную выхода через $\delta_i = p_i - y_i$. Средняя потеря делит этот вклад на число объектов, поэтому входящий градиент выходного logit равен $g_{o_i} = \delta_i / n$. Тогда обратный проход к скрытому logit умножает уже этот входящий градиент на вес выхода и локальную производную tanh:

$$g_{r_i} = g_{o_i} v_1 (1 - h_i^2) = \frac{p_i - y_i}{n} v_1 (1 - h_i^2).$$

Именно этот продукт автоматически возникает из трёх сохранённых узлов: BCE, sigmoid и tanh. Автоматический обратный проход затем умножает g_{r_i} на каждую расширенную координату для скрытых весов; при нулевом индексе это умножение на 1. Проверим все параметры модели на задаче AND, где один скрытый нейрон достаточен.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: and_gradients
AND_INPUTS = [
    [0.0, 0.0],
    [0.0, 1.0],
    [1.0, 0.0],
    [1.0, 1.0],
]
AND_TARGETS = [0, 0, 0, 1]

gradient_and = OneHiddenBinaryClassifier(
    hidden_weights=[0.0, 0.1, -0.2],
    output_weights=[0.0, 0.3],
)
and_loss = mean_binary_cross_entropy(gradient_and, AND_INPUTS, AND_TARGETS)
```

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

```
and_loss.backward()

for parameter in gradient_and.parameters():
    expected_gradient = numerical_gradient(
        gradient_and,
        parameter,
        AND_INPUTS,
        AND_TARGETS,
    )
    assert_close(parameter.grad, expected_gradient, tolerance=1e-5)

assert_close(gradient_and.hidden_weights[1].grad, 0.0)
assert any(abs(parameter.grad) > 1e-8 for parameter in gradient_and.parameters())
print("BCE для AND до обучения:", and_loss.data)
print("Градиенты AND:", [parameter.grad for parameter in gradient_and.parameters()])
```

При этой фиксированной инициализации градиент `hidden_weights[1]` получается равным 0: вклады четырёх объектов AND в первый реальный входной вес точно компенсируются. Индекс 0 теперь занят весом смещения. Это не пропущенный путь обратного прохода — центральная разность в той же ячейке подтверждает ноль. Остальные параметры имеют ненулевые градиенты, а после обучения проверка ниже требует ненулевой градиент у каждого параметра последней эпохи.

Обучение AND

Выполните фиксированный воспроизводимый запуск.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: train_and
learning_rate = 0.3
epochs = 300
trained_and = OneHiddenBinaryClassifier(
    hidden_weights=[0.0, 0.1, -0.2],
    output_weights=[0.0, 0.3],
)
and_losses = []

for epoch in range(epochs):
    loss = mean_binary_cross_entropy(trained_and, AND_INPUTS, AND_TARGETS)
    loss.backward()
    and_losses.append(loss.data)

    for parameter in trained_and.parameters():
        parameter.data -= learning_rate * parameter.grad

and_predictions = [trained_and(features).data for features in AND_INPUTS]
and_accuracy = classification_accuracy(trained_and, AND_INPUTS, AND_TARGETS)

assert and_losses[-1] < 0.05, and_losses[-1]
assert and_losses[-1] < and_losses[0]
assert and_accuracy == 1.0, and_accuracy
assert all(abs(parameter.grad) > 1e-8 for parameter in trained_and.parameters())
print("AND: loss в первой эпохе:", and_losses[0])
print("AND: loss в последней эпохе:", and_losses[-1])
print("AND: вероятности:", and_predictions)
print("AND: точность:", and_accuracy)
```


© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Зачем после AND нужен XOR

После AND мы знаем, как обучается композиция, и теперь проверяем выразительность архитектуры. Один скрытый нейрон не делает XOR решаемым. При пороге $p = 0.5$ sigmoid монотонен, а tanh монотонен, поэтому решение по классу зависит от знака одного аффинного выражения $u_0 + u_1 x_{i1} + u_2 x_{i2}$ с возможным изменением знака из-за v_1. Это одна линейная граница. XOR — не прикладная цель этой лабораторной, а минимальный тест того, что для нелинейно разделимой зависимости нужны две разные скрытые характеристики.

Этап 3. Скрытый слой из двух нейронов и XOR

Уже известно: reverse-mode проходит через композицию двух нейронов. Теперь проверяем другое свойство — выразительность архитектуры: два скрытых нейрона строят два разных нелинейных признака, а sigmoid-выход объединяет их в вероятность класса. Наблюдаемый результат этапа: loss XOR станет меньше 0.03, а все четыре метки будут классифицированы верно. В более крупной MLP та же идея повторяется для большего числа вещественных признаков, нейронов и слоёв.

Пусть скрытый слой имеет два нейрона:

$$h_{ih} = \tanh\left(\sum_{\alpha=0}^{2} u_{h\alpha}\tilde{x}_{i\alpha}\right), \qquad h \in \{1, 2\}.$$

Выходной нейрон собирает их состояния:

$$o_i = \sum_{\beta=0}^{2} v_\beta \tilde{h}_{i\beta} = v_0 + v_1 h_{i1} + v_2 h_{i2}, \qquad p_i = \sigma(o_i).$$

Создайте универсальный класс слоя в `xor_model`. В эксперименте он получает ровно два скрытых нейрона.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: xor_model
class HiddenLayerBinaryClassifier:
    def __init__(
        self,
        hidden_layer_weights,
        output_weights,
    ):
        assert len(hidden_layer_weights) > 0, "Нужен хотя бы один скрытый нейрон"
        feature_count = len(hidden_layer_weights[0])
        assert feature_count > 1, "У скрытого нейрона нужны смещение и хотя бы один
        ↪  вход"
        assert all(
            len(weights) == feature_count
            for weights in hidden_layer_weights
        ), "Все скрытые нейроны должны иметь одинаковое число входов"
        assert len(output_weights) == len(hidden_layer_weights) + 1, (
            "Выходу нужны вес смещения и веса всех скрытых состояний"
        )

        self.hidden_weights = [
            [Value(weight) for weight in weights]
            for weights in hidden_layer_weights
        ]
        self.output_weights = [Value(weight) for weight in output_weights]

    def hidden_states(self, features):
        # Каждая строка весов задаёт фиксированный h; список hidden затем сворачивается
        ↪  по beta.
        return [
```

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

```
            affine(weights, features).tanh()
            for weights in self.hidden_weights
        ]

    def __call__(self, features):
        hidden = self.hidden_states(features)
        output_logit = affine(self.output_weights, hidden)
        return sigmoid(output_logit)

    def parameters(self):
        parameters = []

        for weights in self.hidden_weights:
            parameters.extend(weights)

        parameters.extend(self.output_weights)
        return parameters
```

Градиенты XOR до обучения

В этой модели веса выходного нейрона связывают его с разными скрытыми состояниями. Каждый скрытый параметр получает вклад через свой путь к выходу, а `parameters()` собирает ссылки на все девять обучаемых `Value`-объектов.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: xor_gradients
XOR_INPUTS = [
    [0.0, 0.0],
    [0.0, 1.0],
    [1.0, 0.0],
    [1.0, 1.0],
]
XOR_TARGETS = [0, 1, 1, 0]

gradient_xor = HiddenLayerBinaryClassifier(
    hidden_layer_weights=[[0.1, 0.4, -0.3], [-0.1, -0.2, 0.5]],
    output_weights=[0.0, 0.3, -0.4],
)
xor_loss = mean_binary_cross_entropy(gradient_xor, XOR_INPUTS, XOR_TARGETS)
xor_loss.backward()

for parameter in gradient_xor.parameters():
    expected_gradient = numerical_gradient(
        gradient_xor,
        parameter,
        XOR_INPUTS,
        XOR_TARGETS,
    )
    assert_close(parameter.grad, expected_gradient, tolerance=1e-5)

assert len(gradient_xor.parameters()) == 9
assert any(abs(parameter.grad) > 1e-8 for parameter in gradient_xor.parameters())
print("ВСЕ для XOR до обучения:", xor_loss.data)
print("Градиенты XOR:", [parameter.grad for parameter in gradient_xor.parameters()])
```

Обратите внимание: у двух скрытых нейронов по три веса, включая нулевой вес смещения, а у выхода три веса, включая его нулевой вес. Всего $2 \cdot 3 + 3 = 9$ параметров.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Обучение XOR

В ячейке `train_xor` запуск фиксирован: не меняйте начальные параметры, шаг или число эпох до прохождения проверок. Разные начальные веса скрытых нейронов важны: если они полностью совпадают, оба нейрона будут получать одинаковые градиенты и останутся одинаковыми.

```
# VECTOR_BINARY_CLASSIFICATION_ESSENTIALS_CELL: train_xor
learning_rate = 0.5
epochs = 500
trained_xor = HiddenLayerBinaryClassifier(
    hidden_layer_weights=[[0.1, 0.4, -0.3], [-0.1, -0.2, 0.5]],
    output_weights=[0.0, 0.3, -0.4],
)
xor_losses = []

for epoch in range(epochs):
    loss = mean_binary_cross_entropy(trained_xor, XOR_INPUTS, XOR_TARGETS)
    loss.backward()
    xor_losses.append(loss.data)

    for parameter in trained_xor.parameters():
        parameter.data -= learning_rate * parameter.grad

xor_predictions = [trained_xor(features).data for features in XOR_INPUTS]
xor_accuracy = classification_accuracy(trained_xor, XOR_INPUTS, XOR_TARGETS)

assert xor_losses[-1] < 0.03, xor_losses[-1]
assert xor_losses[-1] < xor_losses[0]
assert xor_accuracy == 1.0, xor_accuracy
assert all(abs(parameter.data) > 0.1 for parameter in trained_xor.parameters())
print("XOR: loss в первой эпохе:", xor_losses[0])
print("XOR: loss в последней эпохе:", xor_losses[-1])
print("XOR: вероятности:", xor_predictions)
print("XOR: точность:", xor_accuracy)
```

Диагностика

После успешного запуска notebook должен показать три понятных результата:

- OR: loss меньше 0.1, все четыре метки классифицированы верно одним sigmoid-нейроном;
- AND: loss меньше 0.05, градиенты прошли через tanh скрытого нейрона и sigmoid выхода;
- XOR: loss меньше 0.03, все четыре метки классифицированы верно слоем из двух скрытых нейронов.

Эти результаты не означают, что модель предназначена для развёртывания как классификатор логических функций. Они подтверждают три свойства, которые затем сохраняются в MLP на реальных данных: обучение линейной границы, прохождение градиента через композицию и построение нескольких нелинейных признаков.

Если проверка не проходит, проверьте следующие причины в указанном порядке:

- все ячейки запускались сверху вниз после последнего изменения `Value`;
- длина каждого сырого вектора признаков на единицу меньше длины соответствующего списка весов: первый вес относится к постоянной координате 1;
- sigmoid применяется только после выходного logit, а tanh только в скрытом слое;
- `loss.backward()` вызывается до обновления `parameter.data`;
- после каждого шага используется `- learning_rate * parameter.grad`, а не знак `+`;
- метка извлекается порогом `0.5`, а не сравнением с нулём;
- два скрытых нейрона XOR имеют разные начальные параметры.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Что намеренно не реализуется

Эта работа не вводит тензоры, матрицы, broadcasting, minibatch-векторизацию, многоклассовый softmax, регуляризацию или готовые классы PyTorch. Вектор признаков разворачивается обычным циклом Python, чтобы связь между формулой, скалярным вычислительным графом и накоплением градиентов оставалась наблюдаемой.

Чек-лист перед сдачей

- ☐ Выполнены все `assert` без ошибок.
- ☐ Центральная разность совпала с autograd для OR, AND и XOR.
- ☐ Объяснено, почему `features` не входят в `parameters()`.
- ☐ Показано, где один параметр используется всеми объектами выборки.
- ☐ Получены требуемые loss и accuracy для OR, AND и XOR.
- ☐ Объяснено, почему одного скрытого нейрона недостаточно для XOR.

Вопросы для самопроверки

1. Какие скалярные вершины создаёт `affine` для двух признаков, включая путь $w_0 \cdot 1$?
2. Почему одна ссылка на `weights[0]` даёт сумму градиентных вкладов от всех объектов датасета?
3. Какой узел хранит вероятность класса 1: скрытый tanh-узел или выход sigmoid?
4. Почему binary cross-entropy совместно с sigmoid даёт для выходного logit градиент $p - y$?
5. Что нарушится, если обновить `parameter.data` до `loss.backward()`?
6. Почему два одинаково инициализированных скрытых нейрона не приобретают разные роли?
7. Почему XOR в этой лабораторной является тестом выразительности, а не прикладной целевой задачей?

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/